OSS-CRS:
Open Framework for Cyber Reasoning Systems and Agentic Security

Andrew Chin
Team Atlanta / Georgia Institute of Technology

Modularizing Agentic Security

Cyber Reasoning Systems and **agentic security** are extremely effective, but such systems remain **monolithic**, **tightly coupled with infrastructure**, and **hard to compose**.

- **Resource management** makes parallelization and composition possible
- **Modularization** via a standardized interface allows for interoperability
- **Flexible deployment matters**: security-critical projects can't always call out to someone else's cloud, and operators may not have the same resources as original authors

What is a Cyber Reasoning System?

A **Cyber Reasoning System (CRS)** is an **evidence-producing system** that participates in an end-to-end security operation.

In the setting of the AI Cyber Challenge, a CRS **finds** and **fixes** vulnerabilities.

- **Explores**: navigates the codebase to identify candidate weaknesses.
- **Reproduces**: converts a claim into an executable proof-of-vulnerability.
- **Diagnoses**: pins the root cause and threat context.
- **Remediates**: proposes a patch that a verifier can independently rerun.
- Techniques span pure fuzzing, static analysis, agents, or any hybrid.

DARPA + ARPA-H

AIxCC
AI CYBER CHALLENGE

WHAT IS AIxCC?

- ➔ A competition that rewards autonomous systems that *find* and *patch* vulnerabilities in source code — **Cyber Reasoning Systems (CRS)**.
- ➔ The challenges are well-known open-source projects.
- ➔ The vulnerabilities are realistic or real.
- ➔ Patching is worth more than finding.
- ➔ Code and data will be released open source.

black hat

AUGUST 2023

OPEN TRACK AND SMALL BUSINESS TRACK SUBMISSIONS

DEFCON

AUGUST 2024

SEMIFINAL COMPETITION

Top 7 teams $2 million each

DEFCON

AUGUST 2025

FINAL COMPETITION

Winners announced

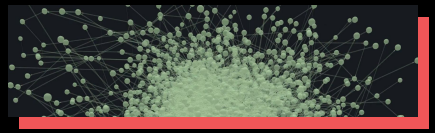
1ST: $4 MILLION

2ND: $3 MILLION

3RD: $1.5 MILLION

Google ANTHROP\C OpenAI Microsoft THE LINUX FOUNDATION OpenSSF OPEN SOURCE SECURITY FOUNDATION

What counts for finals?

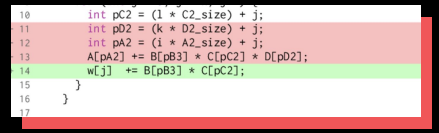


Proof-Of-Vulnerability (POV)

- Input data to reproduce vulnerability crash in harness

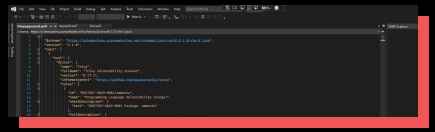
PATCH

- Unified diff source code fix for vulnerabilities



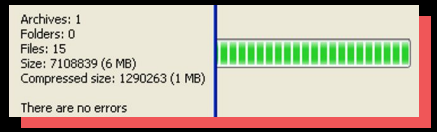
DELTA SCAN

- Challenge analyzing base code plus applied diff changes



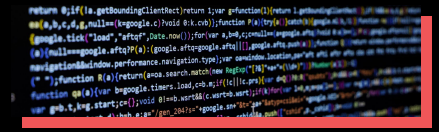
SARIF Assessment

- Structured reporting format for vulnerability details



BUNDLE

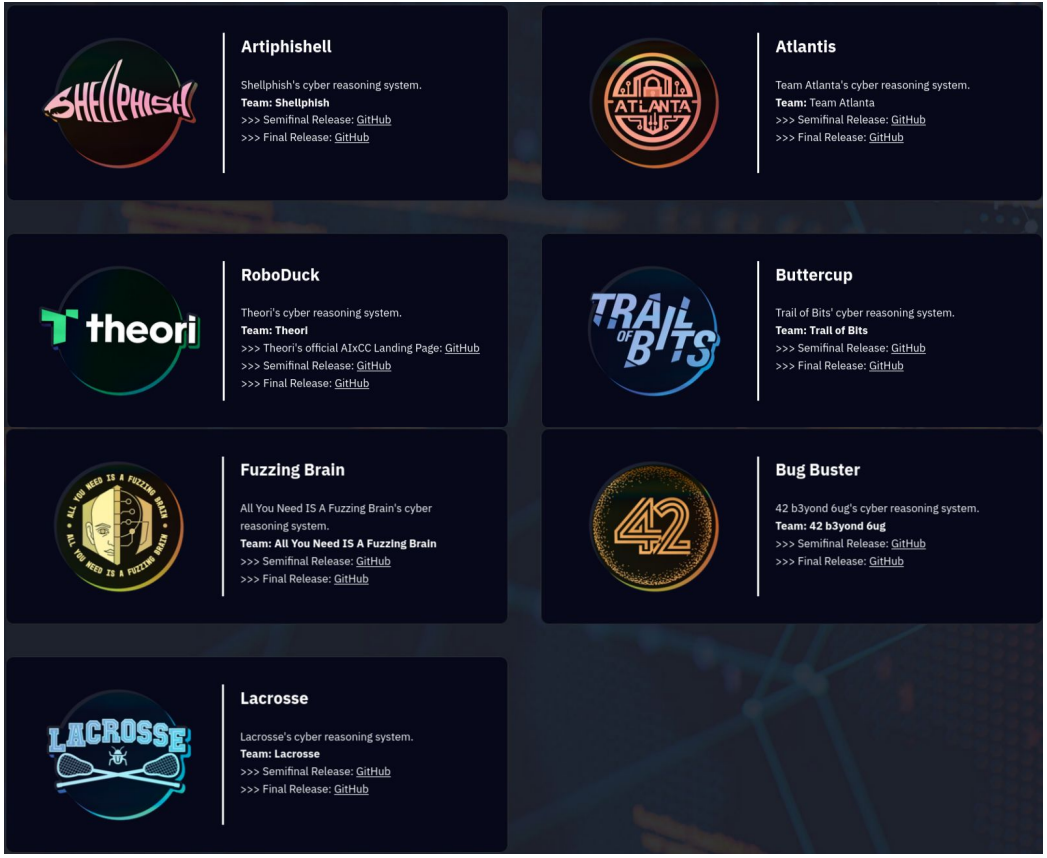
- Grouping of related PoV, patch, and SARIF submissions



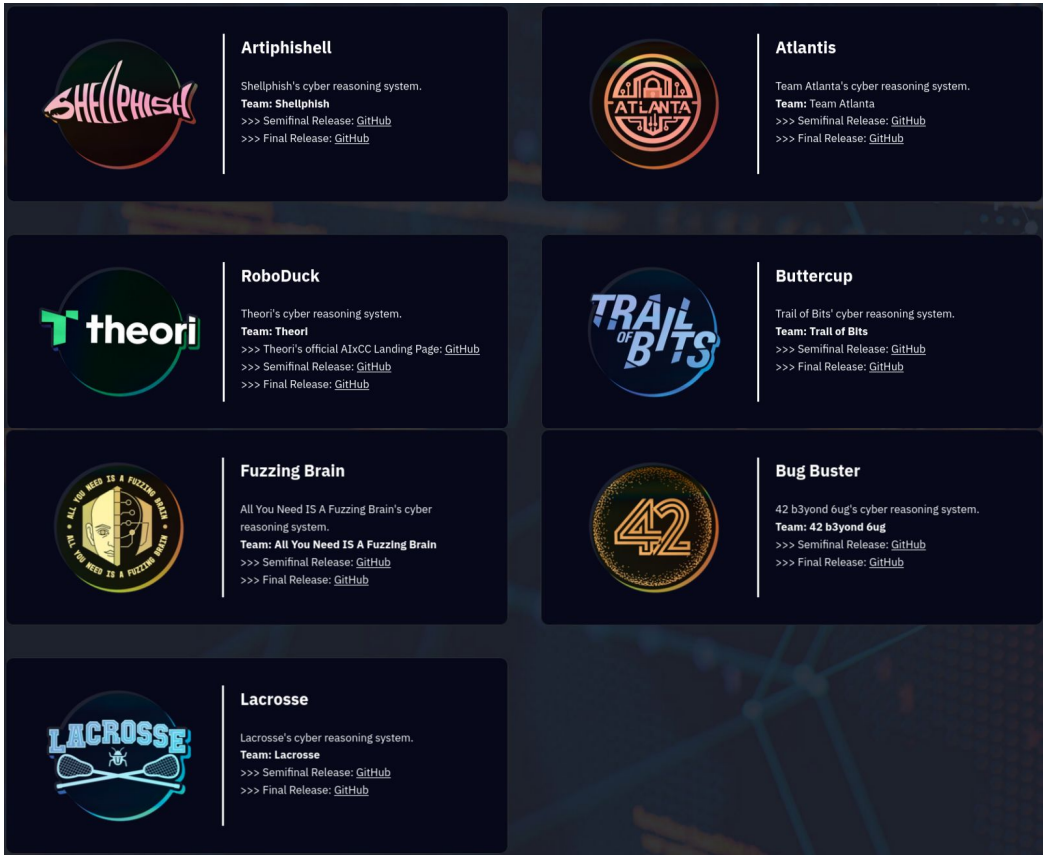
FULL SCAN

- Challenge analyzing entire code base

Open Sourced!



Open Sourced! But largely unmaintained…



No Activity

Archived

Active

No Activity

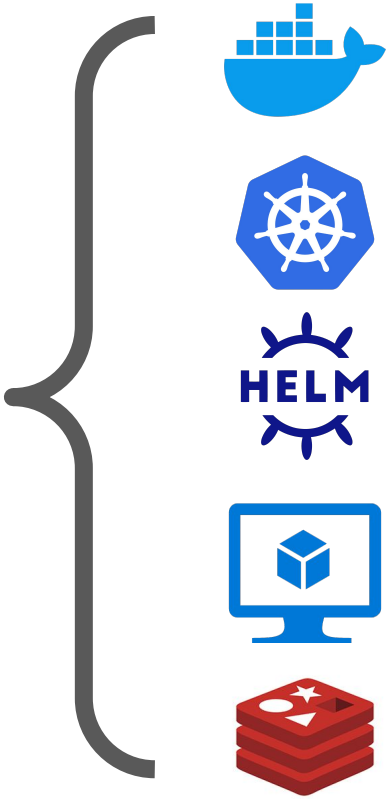
Archived

Active

No Activity

Running All Finalist CRSs

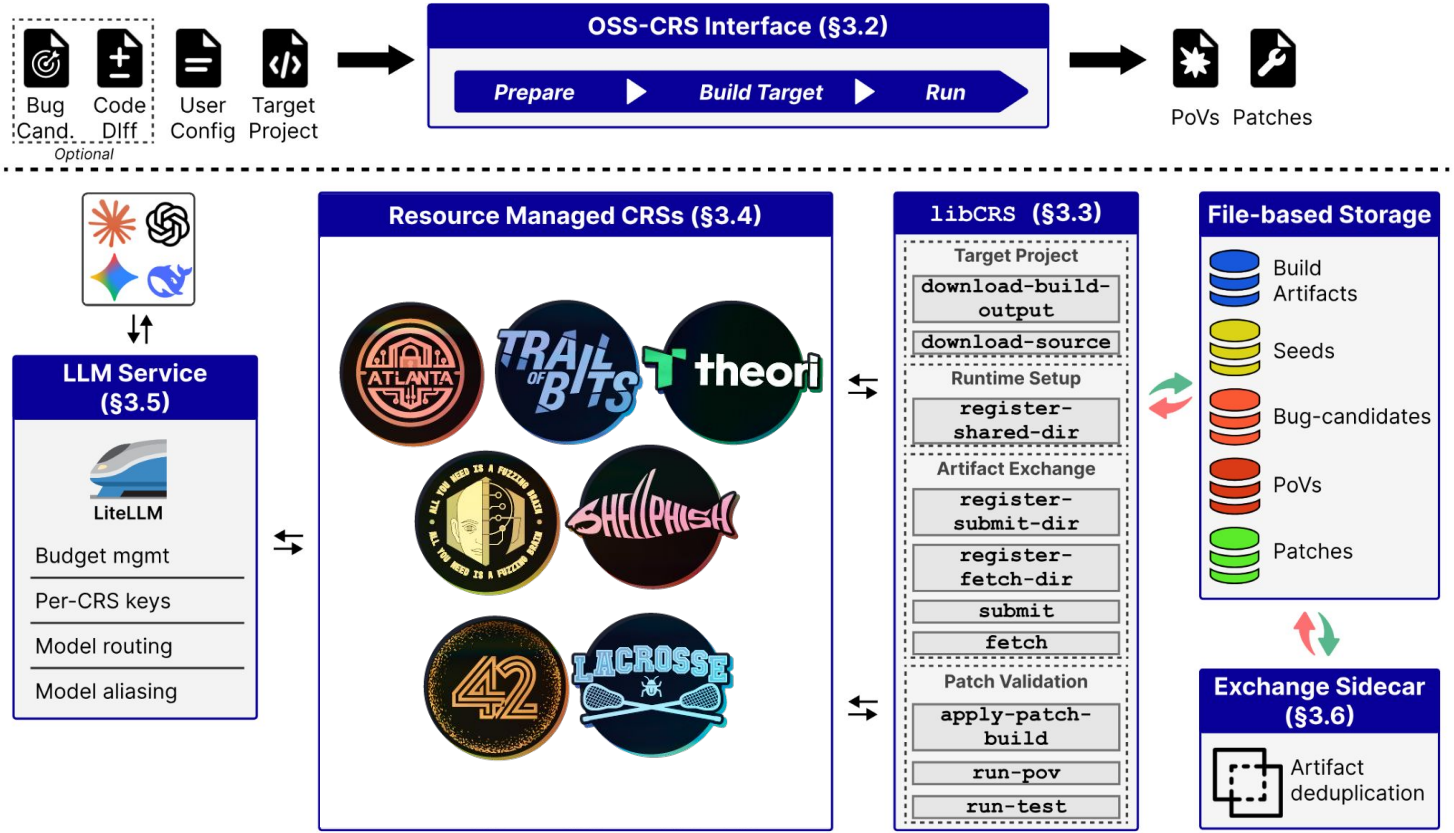
- ✗ Cloud Lock-In
- ✗ Infrastructure Duplication
- ✗ Monolithic Design



Limitations of Monoliths

- **Hard to preserve**: systems bit-rot once their campaign ends.
- **Hard to isolate and reuse**: techniques stay locked inside their stack.
- **Hard to compare**: no benchmarks for security techniques.
- **Hard to improve component-by-component**: you replace the whole thing or nothing.

OSS-CRS: Open CRS Standard & Framework



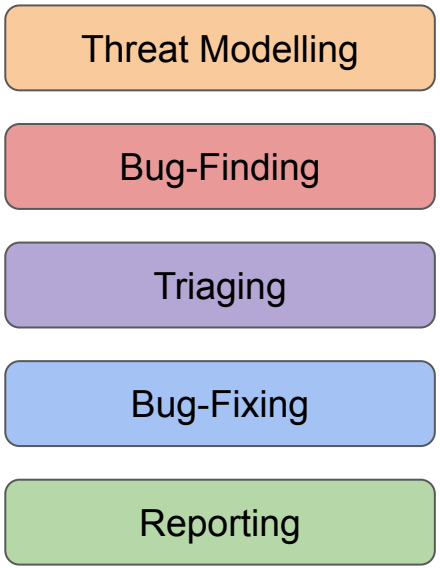
Georgia Tech

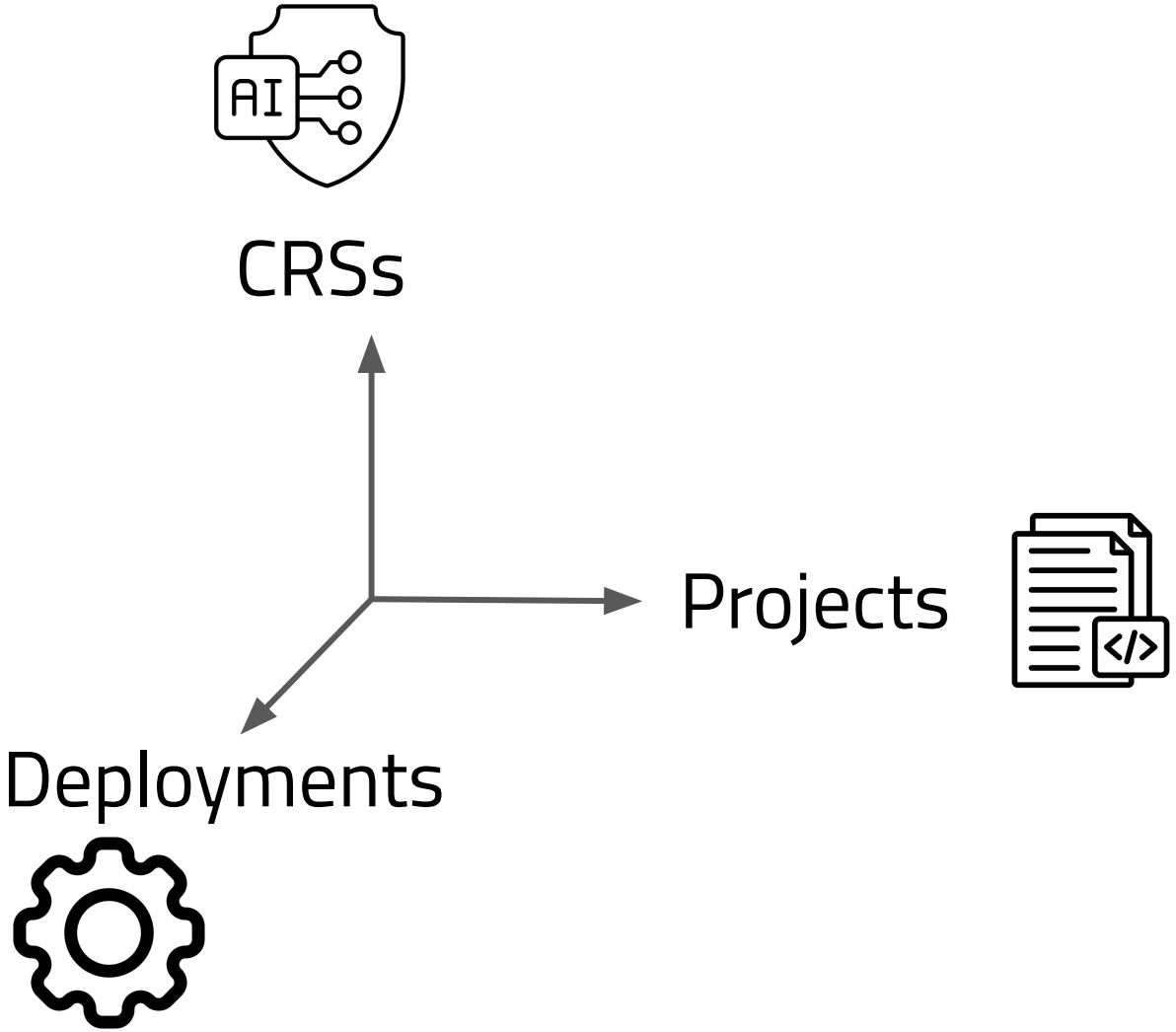
OpenSSF
OPEN SOURCE SECURITY FOUNDATION

Stages in the Security Pipeline

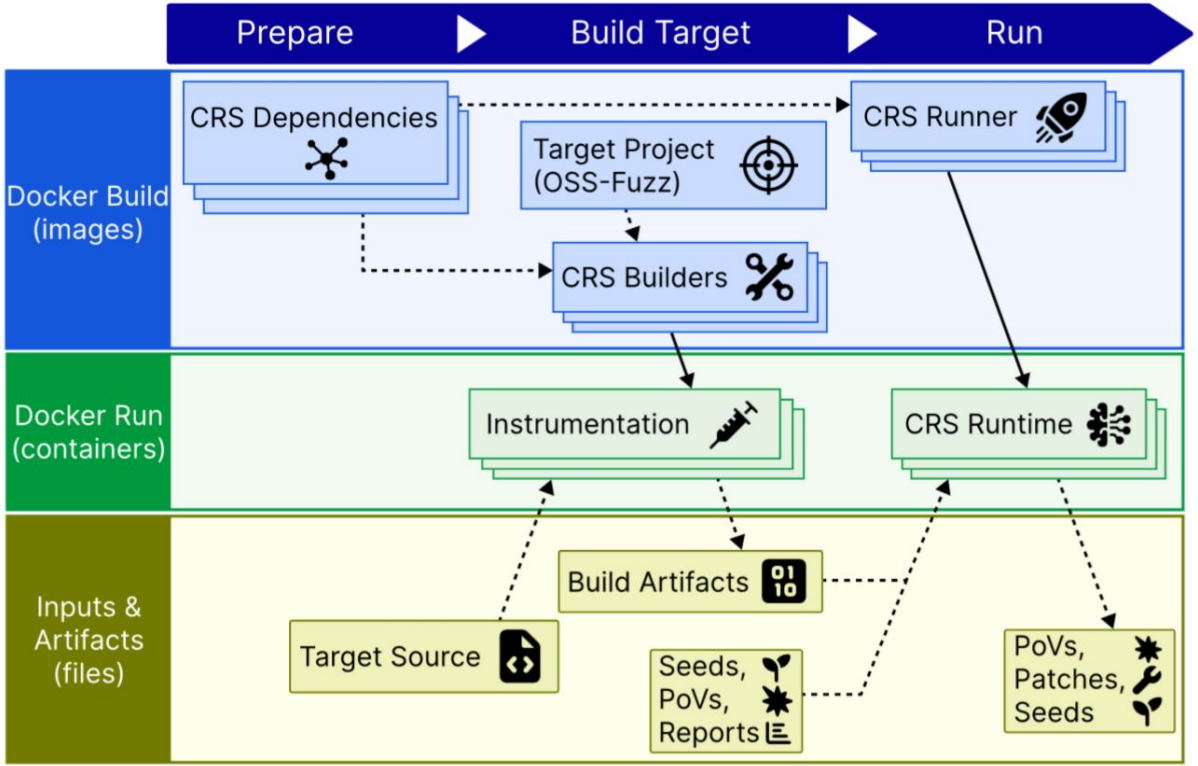
OSS-CRS should

- Enable techniques to cover the whole end-to-end security pipeline
- Define clear boundaries for the stages
- Allow the interchangeability of techniques at any particular stage
- Be flexible to support future additions or enhancements to the pipeline

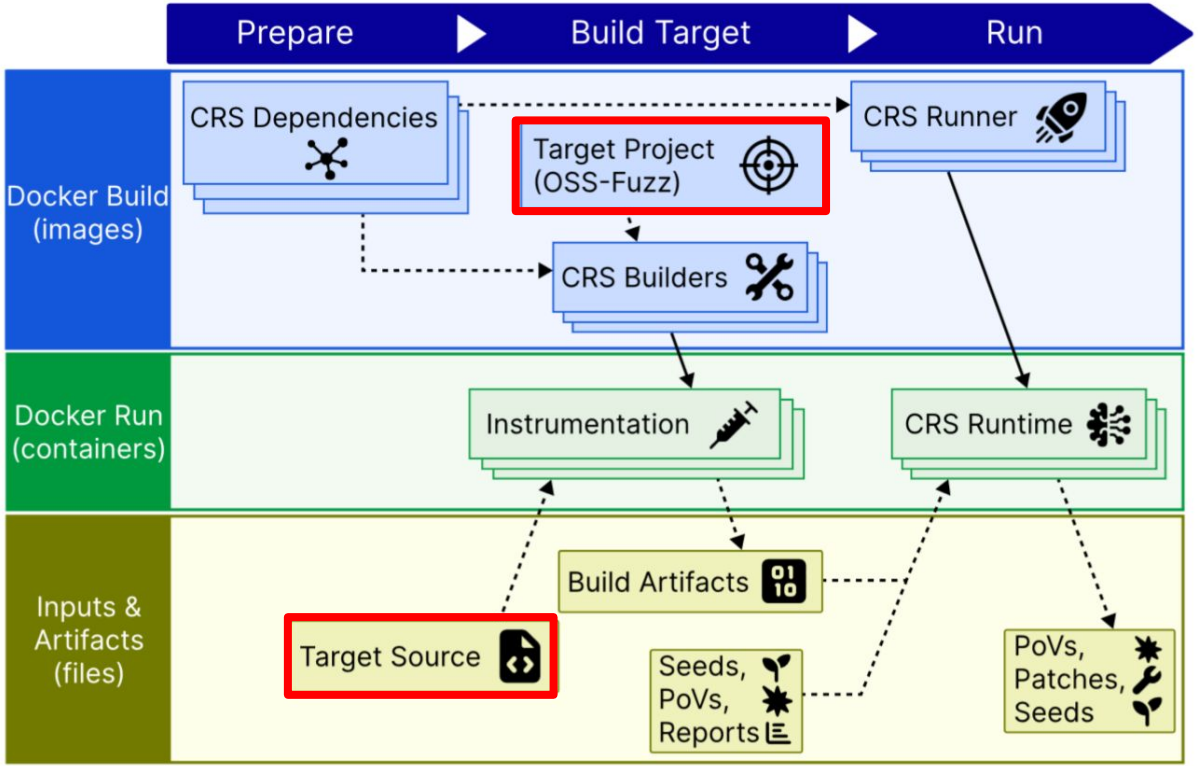



AI
CRSs
Projects
Deployments

Stages


Prepare
Build Target
Run
Docker Build (images)
CRS Dependencies
Target Project (OSS-Fuzz)
CRS Runner
CRS Builders
Docker Run (containers)
Instrumentation
CRS Runtime
Inputs & Artifacts (files)
Build Artifacts
Target Source
Seeds, PoVs, Reports
PoVs, Patches, Seeds

Target Projects



Standardized Environment

OSS-CRS builds on 1000+ OSS-Fuzz projects instead of inventing a new schema

Dockerfile

Reproducible environment for Linux distribution, library dependencies, and disk setup

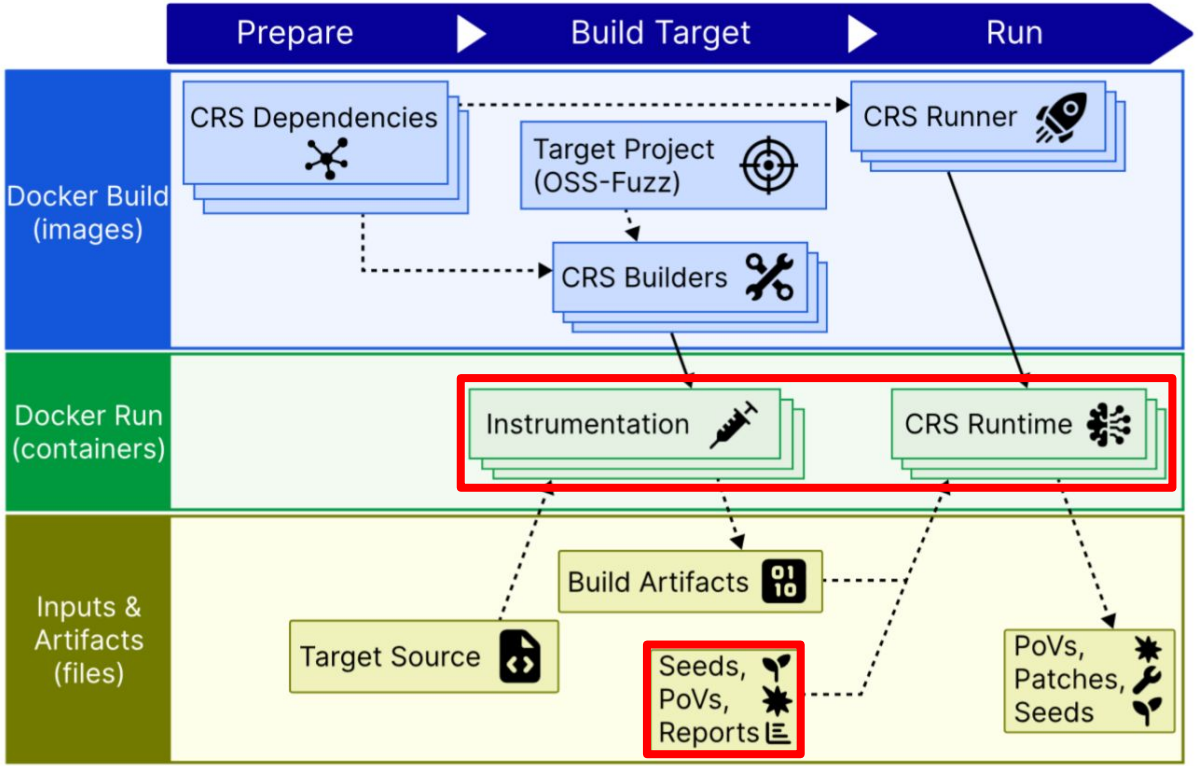
build.sh

Standardized script for compiling the project with sanitizers

Fuzz harnesses

Executable entrypoints that forward attacker-controlled data into public API

Deployments



Resource Management and Configuration

Infrastructure

- Core pinning
- Memory limits

Model

- Provider credentials
- Model routing
- Token + $ budget
- Concurrency caps

Campaign

- Wall-clock timeout
- Parallel CRSs
- Forwarding artifacts from previous runs

LLM First-Class Support via Proxy

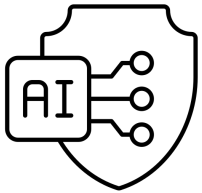
Operators can

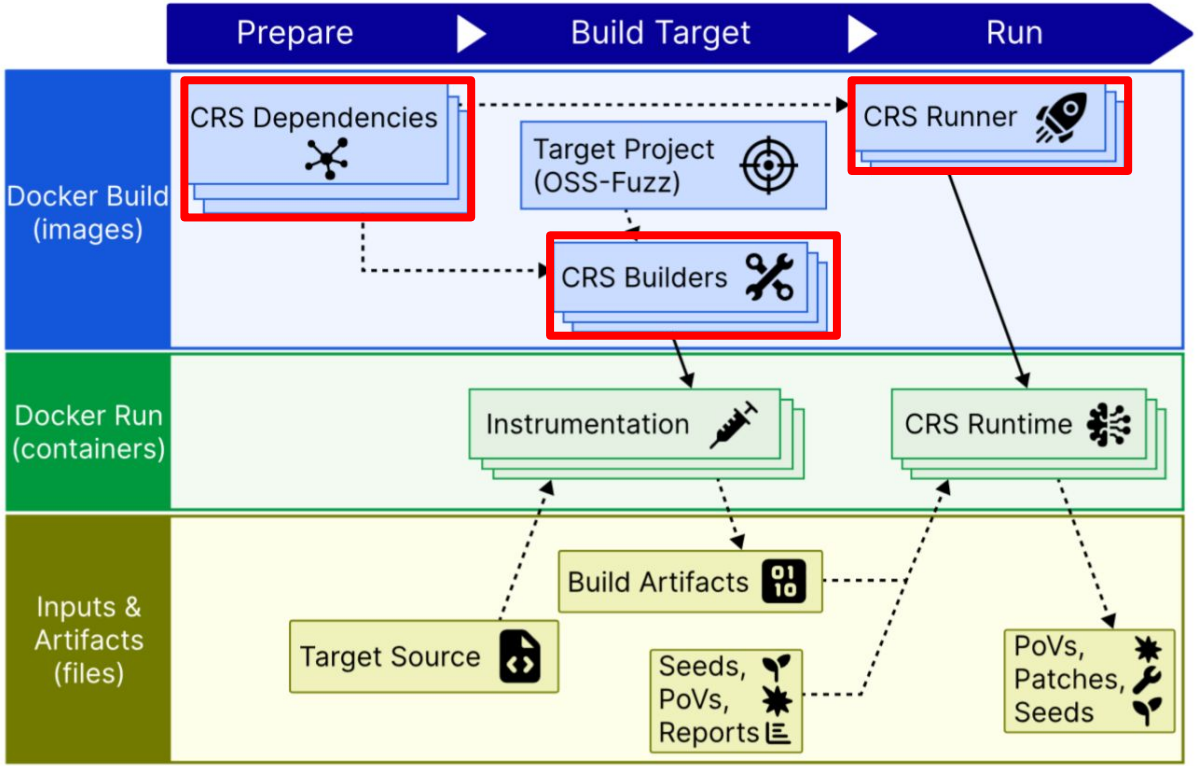
- Bring their own API keys
- Alias “gpt-5.5” → self-hosted vLLM endpoint
- Cap spending per CRS
- Route requests to different providers
- Log calls

CRSs get

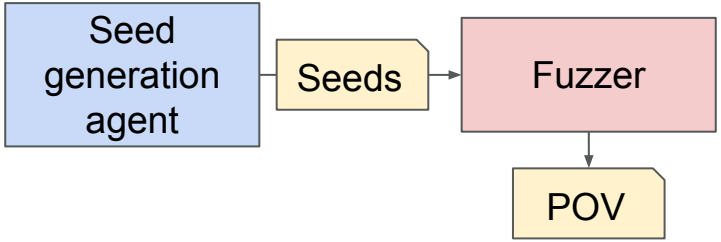
- One base URL and one key
- Consistent model names across campaigns
- Popularly supported endpoints
  (e.g. OpenAI, Anthropic)

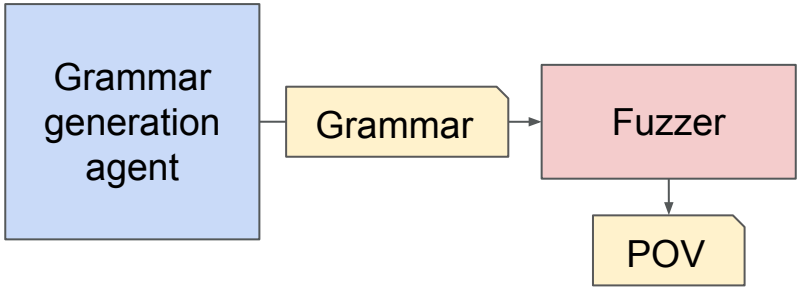
CRSs

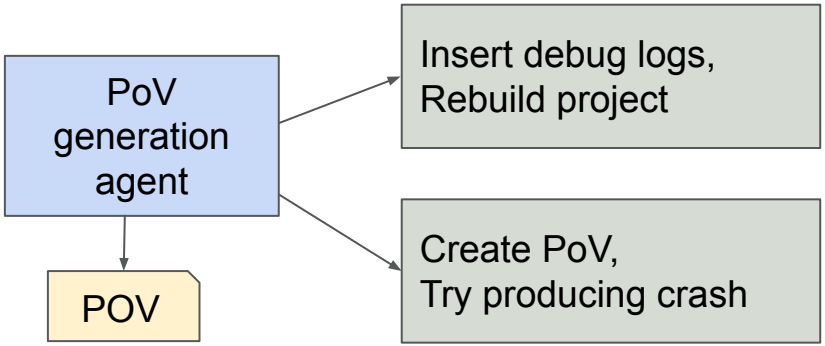

AI

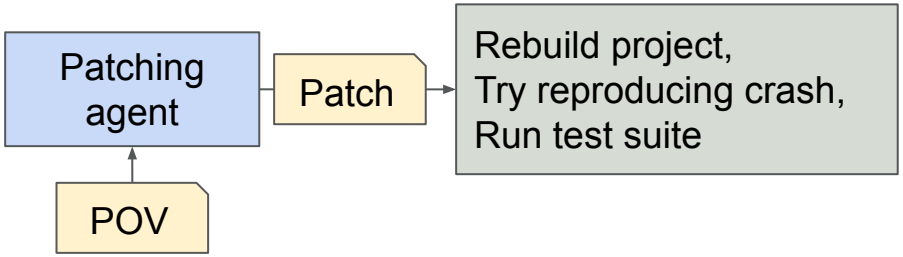

Prepare
Build Target
Run
Docker Build (images)
CRS Dependencies
Target Project (OSS-Fuzz)
CRS Runner
CRS Builders
Docker Run (containers)
Instrumentation
CRS Runtime
Inputs & Artifacts (files)
Build Artifacts
Target Source
Seeds, PoVs, Reports
PoVs, Patches, Seeds

Anatomy of a CRS

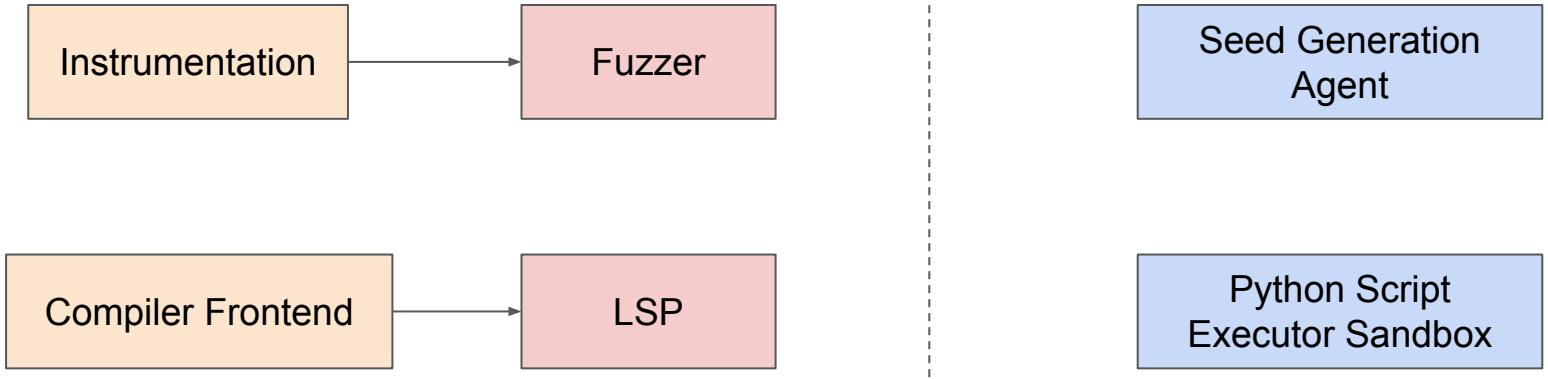








CRS Containers: Dependent v.s. Independent of Target



CRSs dependant on target may be involved with the project's compilation or runtime

CRSs independant on target may only require the static source code of the project

libCRS: CRS-Facing Interface and Helpers

Artifact transfer

- Build outputs – syncing build artifacts between stages
- Submitting findings – PoVs, patches
- Fetching auxiliary data – seeds, bug candidates, PoVs, patches

Helpers for common CRS actions

- Rebuilding the project given a patch – can use custom CRS builder!
- Run PoV to check if buggy behavior reproduces
- Run functionality tests

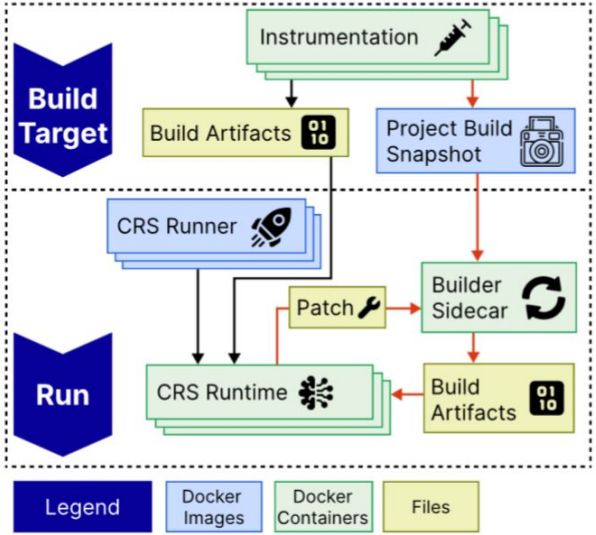
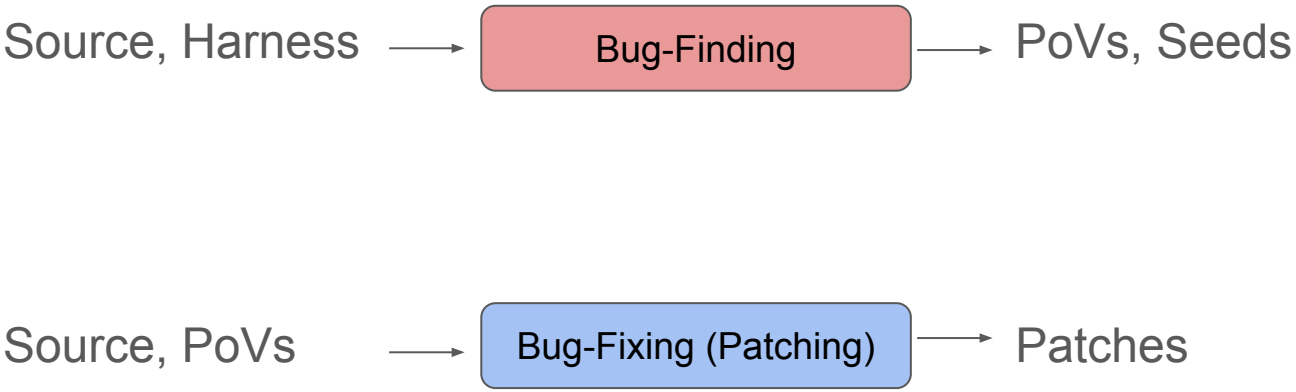
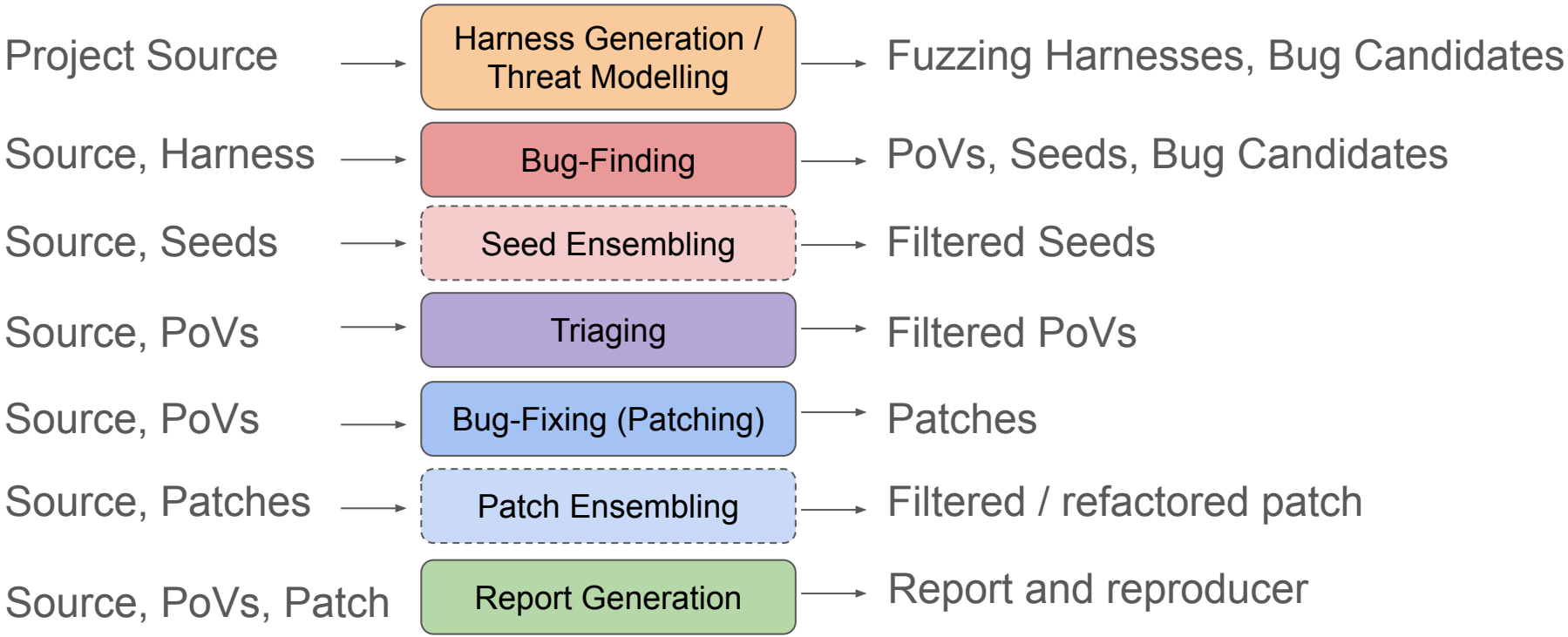


Figure 3: Builder sidecar workflow. The sidecar restores a snapshot of the compiled target, applies the patch diff, and performs an incremental rebuild.

CRS Types (AIxCC)



CRS Types & Artifacts



Artifacts ⇒ CRS Interoperability

Clean I/O Boundaries

Each CRS is defined by what it consumes and emits

Composability

Swap the patcher without touching the bug-finder

Preserve Capability

A useful technique survives outside the stack from which it originated

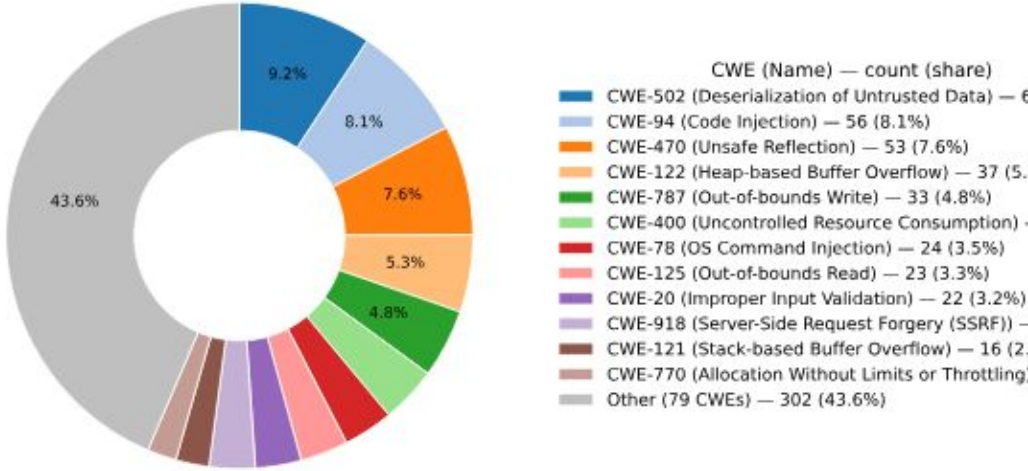
Testable & Comparable

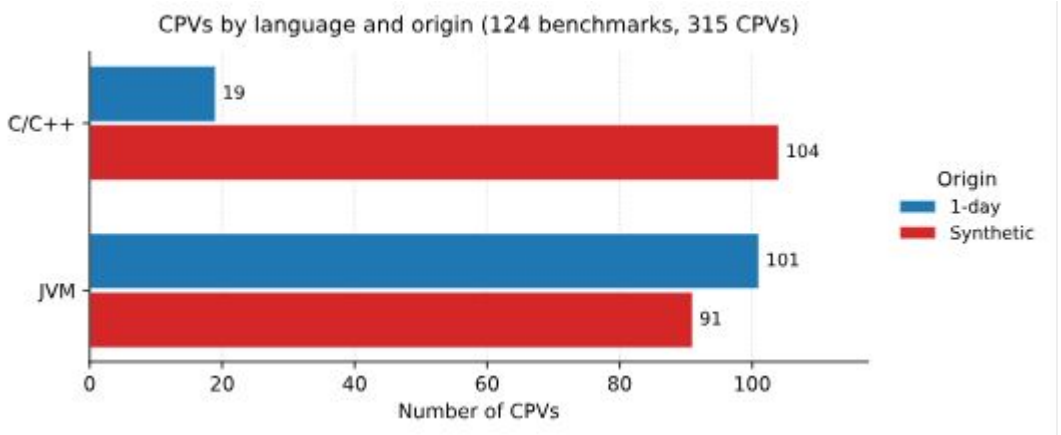
Same artifact and same oracle:
we can measure performance across CRSs

Benchmarks

Benchmarking and testing CRSs

- Use any CRS integrated into OSS-CRS
- Faster evaluation using build snapshots and selective unit testing
- Dataset made from AI Cyber Challenge problem and Team Atlanta internal benchmarks





oss-crs.openssf.org/crsbench

0-days Found and Fixed

Using **Atlantis-Multilang**, a **modern parallel-subagents CRS**, and a **harness-generation CRS**, bugs were found and fixed across 10 OSS projects

- php
- VLC
- wolfSSL
- mosquitto
- memcached
- …

Ongoing Initiatives

MIT Lincoln Lab & DARPA

Offline CRS support for air gapped environments

NYU & CSAW

Mini-AIxCC competition using OSS-CRS as infrastructure

Kubernetes

OSS-CRS Github Actions for CI/CD

Getting Involved

https://oss-crs.openssf.org/

https://github.com/ossf/oss-crs

https://openssf.slack.com/archives/C09FQLYH1RD

(All of these links can be found on https://openssf.org/projects/oss-crs/)

Check out Team Atlanta's website and blog at https://team-atlanta.github.io/

Finally, you can find my contacts on my webpage at https://achin.ca